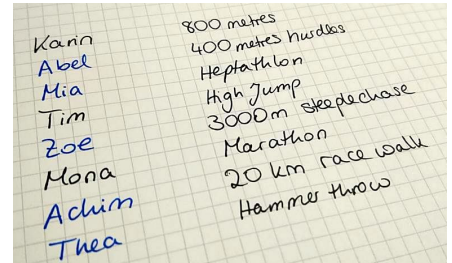


Problem A. Alphabetical Athletes

Source file name: Athletes.c, Athletes.cpp, Athletes.java, Athletes.py
 Input: Standard
 Output: Standard

Anna is organizing the accommodation of the athletes of the German delegation at the Olympic Games. While going over the list of names, she suddenly realizes that some of the names have their letters sorted alphabetically. She wonders if this is simply a funny coincidence or if there are more words in the German language where this is the case.

A word is said to be sorted alphabetically if the letters occur in the same order in the word as in the alphabet, or if they appear in the reverse order as in the alphabet. For simplicity, Anna does not distinguish between upper- and lowercase letters.



Input

The input consists of:

- One line with a string s ($1 \leq |s| \leq 60$), a German word. The word only consists of letters 'a' to 'z' and 'A' to 'Z'. All letters are given in lowercase, except for the first one which is either upper- or lowercase.

Output

Output "yes" if the word is sorted alphabetically, and "no" otherwise.

Example

Input	Output
bekloppt	yes
Ente	no
Rommee	yes

Problem B. Bright Beacons

Source file name: Beacons.c, Beacons.cpp, Beacons.java, Beacons.py
 Input: Standard
 Output: Standard

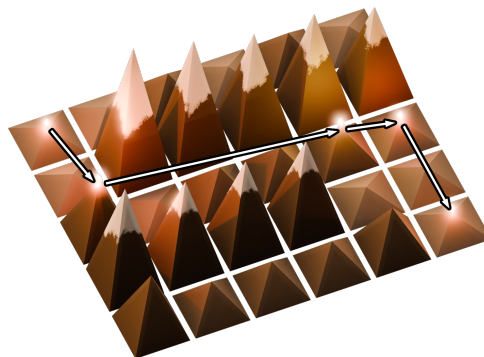
In honour of the Olympic ethos, an archer is faced with a revered task: to light a series of bonfires leading to the Olympic Torch. The archer must establish a network of bonfires, across a grid of hills with heights between 0 and 9, such that they may light the Olympic Torch in the south-east corner of the grid. Bound by ancient tradition, the archer may only shoot in straight lines that follow the cardinal directions of north-south or east-west. You must find the smallest number of additional bonfires needed to create an unbroken chain of bonfires between the starting point and the Olympic Torch.

The rules for establishing visibility between bonfires are:

- Bonfires can be lit when they are within a direct line of sight of another bonfire that is already burning (even if archers may shoot in a parabola, assume that if they cannot see the bonfire, they cannot hit it).
- A mountain is considered to block the line of sight between two bonfires if its peak lies strictly above the straight line between the two peaks of the bonfires in question. When determining if there is a line of sight, assume that each grid cell's terrain height is a peak of zero width in the middle of the cell.
- An archer can only shoot north, west, east or south in a single shot. Shooting in a combination of these directions is not allowed due to the Olympic traditions.

For instance, consider an example with terrain heights in a line like “124”:

- A bonfire placed on terrain height 1 can see a bonfire placed on terrain height 4 because the terrain with height 2 is not tall enough to obstruct the view.
- However, if the terrain heights were “134”, a bonfire on height 1 would not see a bonfire on height 4 due to the peak of height 3 blocking the line of sight.
- In cases such as “222” or “123”, bonfires at the first and last point can see each other since the intermediate heights are not obstructing the direct view.



Path of arrows and lit bonfires in Example 1.

What is the minimum number of additional bonfires you need to place in order to enable signalling from the bonfire at the north-west to the Olympic Torch at the south-east? Assume that the initial bonfire and the Olympic Torch have already been placed.



Input

The input consists of:

- One line with two integers r and c ($2 \leq r, c \leq 100$), the number of rows and the number of columns in the terrain, respectively.
- Then r lines follow, each with c digits. The digits signify the height h ($0 \leq h \leq 9$) of each mountain.

Output

Output the minimum number of additional bonfires needed in order to enable signalling from the bonfire at the north-west corner to the Olympic Torch at the south-east corner.

Example

Input	Output
5 6 112121 198776 412221 766611 422231	3
5 5 25688 59998 69996 89995 88653	5
2 18 01122222322222110 01122222322222110	6

Problem C. Cellar Chase

Source file name: Cellar.c, Cellar.cpp, Cellar.java, Cellar.py
 Input: Standard
 Output: Standard

On Halloween, Quirrell suddenly enters the Great Hall and claims to have seen a troll in the dungeons. The evacuation of the students into their dormitories is started immediately. At the same time, a group of teachers prepares to search the dungeons for the troll.

The dungeons were constructed over a long period of time and are a system of connected corridors. They originally consisted of just one long corridor with an entrance and an exit. Over time, new corridors were added. Each new corridor had its entrance placed on the right-hand wall of an already existing corridor and ended in the same corridor where it started, but further towards the exit. Furthermore, a newly built corridor did only intersect with the corridor containing its entrance and exit. To ensure that nobody gets lost, each corridor contains markers pointing to the corridor's exit.

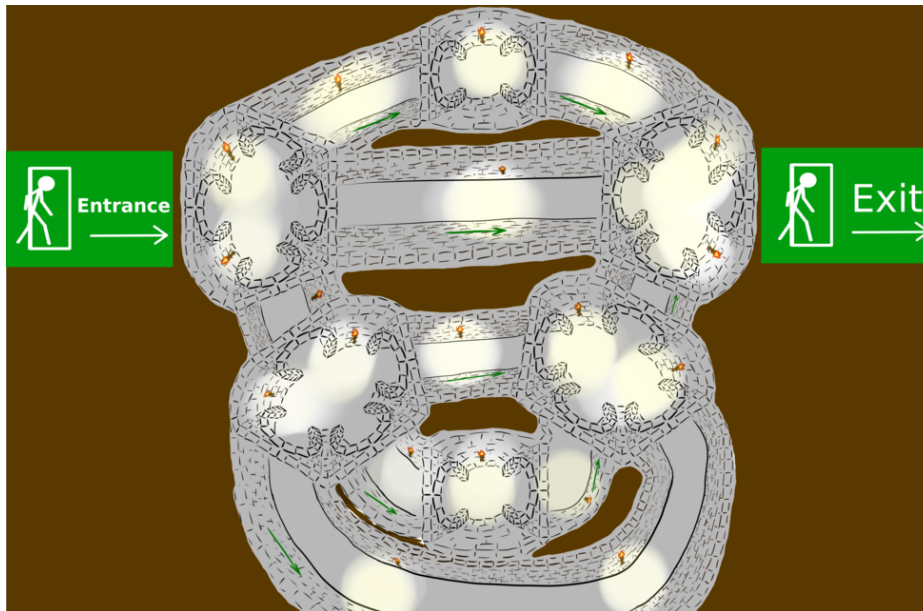


Illustration of the second example.

The teachers have a map of the dungeons and want to use the fact that there is only one exit to their advantage. They lock the exit and start all together at the entrance to the system. To ensure that all of them actually reach the exit, they agree that each of them only moves towards the exit of the current corridor or enters another corridor through its entrance. Of course, they want to avoid that the troll can leave through the entrance or even attack them from behind. Hence, they want to move in such a way that there is never a path between the entrance of the dungeons and the troll which is not blocked by some teacher. Fortunately, each teacher is capable enough to deal with the troll, that is, it is sufficient if it can be guaranteed that at least one teacher encounters the troll. How many teachers have to go to the dungeons?

The system of corridors in the dungeons can also be constructed by composition of systems of corridors. For each such system there are two positions that are defined as the entrance and the exit of the corridor, respectively.

- $()$ denotes just one corridor with an entrance and an exit.
- $(A+B)$ denotes that we append the system B to A, that is, the entrance of A is the new entrance of



the system, the exit of **B** is the new exit and we merge the exit of **A** with the entrance of **B**.

- $(A*B)$ denotes that **A** and **B** are parallel to each other. This means that the entries of **A** and **B** are merged and form the new entrance and the exits of **A** and **B** are merged as well and form the new exit.

Input

The input consist of a single string of length at most 10^6 describing the dungeons as explained above.

Output

Print a single integer, the minimum number of teachers needed to examine the entire dungeons.

Example

Input
$((()+((()*((()+())))))$
Output
2
Input
$(((((()+())*())*(((()+(((()+())*())*()))+()))$
Output
5

Problem D. Document Dimensions

Source file name: Document.c, Document.cpp, Document.java, Document.py
Input: Standard
Output: Standard

Hermione was really proud of her one million word text she wrote for her assignment. She was, until she realized that the text must be handed in on a single piece of paper with limited dimensions. Obviously, she could have just shortened her text, but Hermione decided to go another route. She decided to just copy her text to a new piece of paper, writing a little bit smaller. . . To make this easier, she decided to first change the line breaks in her text such that the sum of the *height* and *width* of the piece of paper is minimized. Given Hermione's text with n words and assuming that each character takes up one unit height and one unit width, what is the minimal *height* plus *width* that can be achieved by inserting line breaks? Note that two words which are on the same line need to be separated by a single space.



Hermione's text. Image by Chris Martin, CC BY-SA Wikimedia

Input

The input consists of:

- One line with a single integer n ($1 \leq n \leq 10^6$), the number of words.
- One line with n space separated words w_i ($1 \leq |w_i| \leq 10^6$), consisting only of lowercase Latin letters.

It is guaranteed that the total length of the text, i.e. the sum of the lengths of the n words, is not greater than 10^6 .

Output

Output a single integer, the sum of the *height* and *width* of the smallest piece of paper the text could fit on.

Example

Input	Output
4 i am lord voldemort	11
10 i solemnly swear that i am up to no good	14

Explanation

These are visualizations of the optimal result.

First testcase: $2 + 9$

```
i am lord
voldemort
```

Second testcase: $4 + 10$

```
i solemnly
swear that
i am up to
no good
```

Problem E. Euroexpress

Source file name: Euroexpress.c, Euroexpress.cpp, Euroexpress.java, Euroexpress.py
 Input: Standard
 Output: Standard

You want to travel to the Olympic Games this year and already decided to take the Eurostar to Paris to be more environmentally friendly. Your next decision is to pick a suitcase for your journey. Upon reading the terms and conditions, you noticed that there is no clear size limit for the suitcase. Instead, they provide various two-dimensional constraints, and your suitcase is compliant if it fits in a box where each side matches one of the aforementioned constraints.



Suitcase size check. Photo by Kenzel2

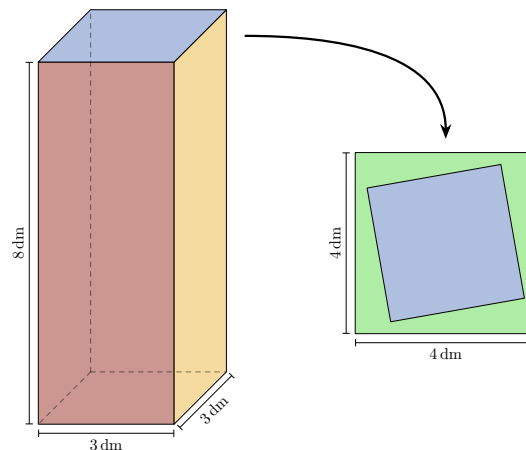


Illustration of Example 2. A suitcase with dimensions $3\text{dm} \times 8\text{dm} \times 3\text{dm}$ fits in a box where each side has either dimension $3\text{dm} \times 8\text{dm}$ or $4\text{dm} \times 4\text{dm}$, i.e. complies either with constraint 3 or 1 of the input.

Since you need to buy a new suitcase anyway, you wonder how much volume could a suitcase have and still be compliant?

Input

The input consists of:

- One line with an integer n ($1 \leq n \leq 2 \cdot 10^5$), the number of constraints.
- n lines, each containing two integers a and b ($1 \leq a, b \leq 10^6$), the dimensions of the constraint in dm.

Output

Output a single integer, the maximum volume of a suitcase that you can carry with you in dm^3 .



Example

Input	Output
3 2 31 5 5 13 3	125
4 4 4 2 15 8 3 20 1	72

Problem F. Football Figurines

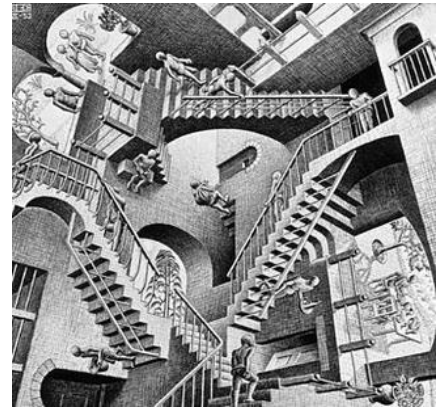
Source file name: Football.c, Football.cpp, Football.java, Football.py
Input: Standard
Output: Standard

Frank is an avid football fan and of course attends the final game live in the Olympic football stadium. It is a huge building, and Frank is impressed by the maze of staircases connecting the n floors.

Frank notices that from each floor there are exactly two staircases going up, one to the floor directly above and one to the floor two levels up (skipping the floor directly above). On each staircase, a volunteer is distributing little figurines to the spectators, giving one figurine to everyone climbing up this staircase.

There may be many different routes between any two floors, where a route consists of a sequence of staircases. Note that, because of the mass of spectators heading to their seats, Frank can only take staircases going up. Frank is wondering how many figurines he can collect when walking up again and again but taking a different route each time. He considers two routes to be different if they differ in at least one staircase.

Given several queries, each consisting of a pair of two floors, find the maximum number of figurines Frank can collect when climbing up all the different routes between those floors.



M.C. Escher, Relativity

Input

The input consists of:

- One line containing two integers n and q ($1 \leq n \leq 10^6$, $1 \leq q \leq 5 \cdot 10^4$), the number of floors and the number of queries, respectively.
- q lines, each containing two integers s and t ($1 \leq s \leq t \leq n$), the numbers of two floors.

The n floors are numbered from 1 to n .

Output

For each query, output the total number of figurines Frank can collect when climbing up all different routes between the two floors. Since this number can be very large, output it modulo $10^9 + 7$.

Example

Input	Output
5 4	0
1 1	1
2 3	3
1 3	15
1 5	
1000000 1	884826434
1 1000000	

Problem G. Genius Gamer

Source file name: Gamer.c, Gamer.cpp, Gamer.java, Gamer.py
 Input: Standard
 Output: Standard

Having had a very interesting day watching this year's Road Cycling Finals, you and your friends want to enjoy the evening in the lobby of your hotel. Unfortunately, nobody brought any games to pass the time. You hope that the hotel provides some games and indeed that's the case. In a rack besides many others, you find a copy of the game Rummikub and you decide with your friends to give it a try.

This game is played with tiles where each tile has a number written in one of four colours. In the hotel's version, there are no jokers and every two tiles either differ in their colour or numerical value. Each player has a set of hidden tiles and there are tiles in a common area. The goal of each player is to place all their tiles into the common area.

In the common area, tiles have to be combined to form groups or runs. A group is composed of at least 3 tiles that all have the same numerical value, but different colours. A run is a combination of at least 3 tiles of the same colour, that have consecutive numerical values. Note that in your version of the game, 1 does not follow 13. So the tiles 12, 13 and 1 in red would not form a valid run.

In each turn players can place their hidden tiles into the common area and combine them with existing groups and runs. To make it more interesting, they can even rearrange the common area freely – without removing any tiles of course. The only restriction is that after the move, the common area should be organized into groups and runs again.

It is your turn now. You ask yourself whether you can place all your hidden tiles into the common area this turn. Write a computer program to help you.

Input

The input consists of:

- One line with an integer n ($1 \leq n \leq 52$), the number of tiles at your disposal.
- n lines, each containing a string c and an integer v ($c \in \{\text{"Red", "Yellow", "Blue", "Black"}\}$, $1 \leq v \leq 13$), indicating that you have the tile with value v and colour c at your disposal.

No combination of colour and value appears more than once in the input.

Output

Output "possible" if you can place all your tiles into the common area in this round and "impossible" otherwise.



The unique solution to Example Input 4.



Example

Input	Output
3 Red 1 Black 1 Yellow 1	possible
8 Red 3 Blue 3 Red 1 Red 2 Black 3 Red 4 Red 5 Yellow 3	possible
7 Red 3 Blue 3 Red 1 Red 2 Black 3 Red 4 Red 5	impossible
21 Red 1 Red 2 Red 3 Red 4 Red 8 Blue 3 Blue 4 Blue 5 Blue 7 Yellow 3 Yellow 4 Yellow 5 Yellow 6 Yellow 7 Yellow 8 Black 3 Black 4 Black 5 Black 6 Black 7 Black 8	possible

Explanation

The unique solution to Example Input 4 is depicted in the image of the story.

Problem H. Haggling over Hours

Source file name: Haggling.c, Haggling.cpp, Haggling.java, Haggling.py
Input: Standard
Output: Standard

Hannah is responsible for organizing the time slots at which the Athletics competitions at the Olympic Games are held. She has finally found one that adheres to the constraints imposed by the Olympic committee and the other sports. Regrettably, someone else is responsible for assigning athletes to the time slots. After talking to a few athletes, she realizes that a bad assignment of athletes to time slots could be unfortunate for some of them.

Therefore, she wants to reduce the maximum number of time slots an athlete could possibly be assigned to by at least one. She wants to do this by removing some of the time slots as this is the easiest way to adjust her plan. Of course, she wants to remove as few time slots as possible. Note that an athlete can only be assigned to several time slots if there is a break of at least one hour in between consecutive ones.



100m hurdles at the 2020 Summer Olympics,
By Bob Ramsak on Wikimedia Commons

Input

The input consists of:

- One line with an integer n ($1 \leq n \leq 10^3$), the number of time slots.
- n lines, each containing two integers a and b ($0 \leq a < b \leq 10^9$), the start and end time of a time slot in hours.

It is guaranteed that no two time slots are identical.

Output

Output the minimum number of time slots which have to be removed such that the maximum number of time slots an athlete could be assigned to decreases by at least one.



Example

Input	Output
5 0 2 1 4 3 6 5 8 7 9	1
6 0 2 1 3 4 6 5 7 8 10 9 11	2
5 0 2 3 6 1 2 1 4 5 6	2

Problem I. Inconspicuous Identity

Source file name: Identity.c, Identity.cpp, Identity.java, Identity.py
Input: Standard
Output: Standard

Last night, Rubeus Hagrid once again visited his giant half-brother Grawp in the forbidden forest. Unfortunately, when Grawp saw Hagrid's pink umbrella, he used it to chase barbaric bowtruckles in the trees. In this chaotic chase, Grawp unfortunately broke Hagrid's umbrella into pieces. Back in his homey hood, Ruben Hagrid wants to assemble a new umbrella. In the past, he only stored the broken pieces of his wand in it. However, he recently realized that members of the non-magic community mostly use umbrellas to protect them against the rain. In order to blend in and hide his true identity whenever he goes shopping to London, Hagrid wants his new umbrella to look exactly like any other umbrella.



Photo by Pascal Robert, Pixabay

The umbrella should consist of a straight wooden handle with 8 metal ribs of length x meters connected symmetrically to its top. Hagrid wants to attach the remaining non-stretchable, rainproof, pink fabric he found in his hood along the full length of the metal ribs, i.e. from the top of the umbrella where all ribs meet down to the very end of each metal rib. If the umbrella is opened all the way, the fabric between two metal ribs forms a isosceles triangle. Depending on the amount of fabric that is used, the umbrella can be opened more or less wide.

Hagrid has up to a square meters of non-stretchable, rainproof, pink fabric available for the umbrella. Since he is quite a corpulent man, he wonders what the maximum area is that the umbrella can keep dry if the rain falls perpendicular to the ground and there is absolutely no wind?

Input

The input consists of:

- One line with two real numbers a and x where
 - a ($0 < a \leq 10$) is the amount (in square meters) of non-stretchable, rainproof fabric that is available for the production of a single umbrella.
 - x ($0 < x \leq 5$) is the length (in meters) of a metal stick.

All real values are given with at most three decimal places.

Output

Print a single real value, the maximum area (in square meters) that can be kept dry. Your solution is considered correct if the relative and absolute error is less than 10^{-4} .

Example

Input	Output
10.000 0.500	0.7071067812
10.000 5.000	1.2101397319

Problem J. Jog in the Fog

Source file name: Jog.c, Jog.cpp, Jog.java, Jog.py

Input: Standard

Output: Standard

On the night before the big event, your fellow distance runner Jesse is running a few final laps on the practice track near the Olympic village. You want to join them; however, it is so foggy that you can not even see one metre ahead.

You know the running route of Jesse, which they will repeat in loops. Using an optimal strategy, how long will you need on average to reach Jesse?

Both Jesse and you move in steps on a 2D grid, one grid cell at a time. You both move either up, down, left, or right a full cell. It takes one second to move one cell. Jesse will always be moving along their fixed route in a loop, and this route will visit no cell multiple times, except after they start looping their route. When you start your attempt, Jesse could be on any step of their route, uniformly at random. You may wait in a cell for 1 second instead of moving. If you and Jesse are in neighbouring cells and move towards each other, you will meet after 0.5 seconds.

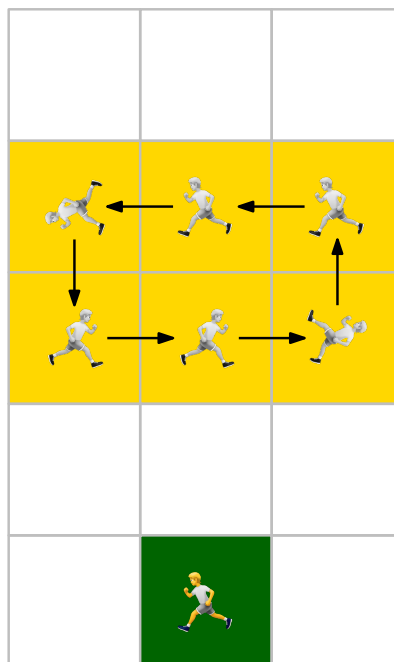


Illustration of Example Input 2, where the optimal expected time is 3.25 seconds.

Input

The input consists of:

- One line with three integers x , y , and n ($0 \leq x \leq 10^9$, $0 \leq y \leq 10^9$, $2 \leq n \leq 10^5$), describing your initial position (x, y) and the length n of Jesse's looping route.
- n lines, the i th of which contains two integers x_i and y_i ($0 \leq x_i \leq 10^9$, $0 \leq y_i \leq 10^9$), describing the i th position of Jesse's route.

All neighbouring steps of Jesse are guaranteed to be of distance exactly 1, so Jesse will always be moving one step up, down, left, or right. This guarantee also holds for the last and first position, allowing Jesse to loop their route. Additionally, no two positions in this route are the same. See Figure 4 for an example.



Output

Output the expected time in seconds to reach Jesse assuming an optimal strategy. Your answer should have an absolute or relative error of at most 10^{-6} .

Example

Input	Output
2 1 2 4 4 4 5	5.25
1 0 6 0 2 1 2 2 2 2 3 1 3 0 3	3.25

Problem K. Keeping Keys

Source file name: Keys.c, Keys.cpp, Keys.java, Keys.py
Input: Standard
Output: Standard

At the Olympic Media Centre, an eco-friendly printer is provided to print schedules and results. In order to reduce the ink consumption, it's meant to charge 1 cent for each lowercase letter and 2 cents for uppercase letters. But you've found a glitch! It seems to not charge per letter, but per key pressed on the attached keyboard. As a skilled typist, you realize that you can hold a key to type any number of the corresponding character for a single cent. Furthermore, you realize that the cost for capital letters is implemented by using the cost of pressing the character's key and the cost of pressing the shift key. Finally, a space character can be typed by pressing the space bar, and this can be done even while holding shift (pressing the space bar costs 1 cent as well).



Example 5 printed using the eco-friendly printer.

What is the minimum cost in cents you will be charged by the printer system if you use the printer as efficiently as possible? You can hold and release keys in any combination you want.

Input

The input consists of:

- One line with a string s ($1 \leq |s| \leq 1000$), consisting solely of the characters 'a' to 'z' and 'A' to 'Z' as well as spaces. The line neither starts nor ends with a space (but the input ends with a line break, which does not need to be paid).

Output

Output the minimum cost in cents needed to print the text.

Example

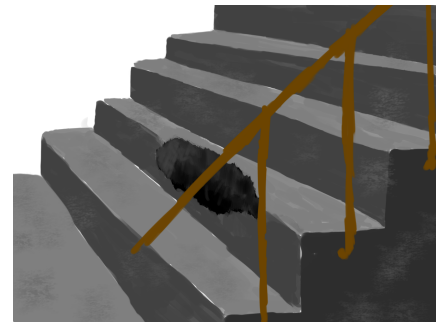
Input	Output
Hello	5
AAAAAAAAAA	2
Buy Llamas	11
A AaA	5
The RUNNERS set new world records	34

Problem L. Longbottom Leap

Source file name: Longbottom.c, Longbottom.cpp, Longbottom.java, Longbottom.py
Input: Standard
Output: Standard

During his seventh year in Hogwarts, Neville Longbottom reactivated Dumbledore's Army with Ginny and Luna, and replaced Harry as their leader in order to fight against Voldemort's takeover. As he did so often lately, this night Neville sneaked into the restricted section of the library to find new ways to inconvenience Snape and his fellow Death Eaters and Slytherins.

Among the books he discovered during his previous visit, one in particular caught his attention. It contains a spell which supposedly fixes the vanishing steps in certain staircases that had plagued Neville during his entire time in Hogwarts and are now used as an opportunity to ridicule and belittle the poor first-years more than ever. Willing to try this spell and put an end to this, he decided to take the book this time.



Luckily, the spell allows fixing all the vanishing steps in an arbitrarily long sequence of consecutive steps at once. However, the spell gets longer and longer the more consecutive steps are considered. For the spell's shortest possible form, this sequence can contain up to 32 consecutive steps. In order to increase this number, the word `long` can be appended an arbitrary number of times to the front of the spell, doubling the maximum number of consecutive steps every time. For instance, `long long` and `long long long` can be applied to sequences of consecutive steps of length 64 and 128, respectively.

Unfortunately, the new regime under Snape greatly strengthened the safety measures, so Neville should return the book as soon as possible so that they won't notice that someone sneaks into the restricted section and close the last remaining loopholes in their security system. Because the spell requires the casting wizard to stand close to the first step of the considered sequence, it would take too long to apply it multiple times. Furthermore, he of course wants to keep the spell itself as short as possible.

Neville already knows the sequence of consecutive steps he wants to fix using the spell and also knows which of these steps need fixing, but is not sure what the shortest sufficient form of the spell is. Can you help him?

Input

The input consists of:

- One line containing a single string s describing the sequence of steps Neville wants to apply the spell to. Each character in s is either 0 or 1, where a 1 indicates that the respective step is a vanishing step. s contains at most 10^6 characters and no leading or trailing zeroes.

Output

Output the shortest sufficient form of the spell.

Example

Input	Output
1	long
10001110101011110100011010101111	long
10000000000000000000000000000001	long long

Problem M. Montage Matrix

Source file name: Montage.c, Montage.cpp, Montage.java, Montage.py
Input: Standard
Output: Standard

You have been hired by the organizing committee of the Olympic Games to take pictures of the event. Of course, you focus on the most important task: taking pictures of the winners!

Unfortunately, you quickly realize that for some competitions, it is quite complicated to take the perfect shot. Especially troubling are team sports like field hockey with a large number of players, both on the field and later on the photo.

The main challenge with so many people is to fit everyone on the photo! After all, so many people do not fit in the same row without anybody being cut off the photo. Therefore, each sports team must form multiple rows with at most w people per row.

Of course, for a perfect picture, all players must be visible on the photo and not be blocked by someone standing in front of them. This means that for each person on the photo, only people of strictly smaller height may stand in front of them.

However, you are unsure whether this is even possible for all teams taking part in the competition. Determine whether a given team can be correctly arranged for the photo.



Argentinian Field Hockey Silver Medal winners 2020. By Secretaría de Deportes on Wikimedia

Input

The input consists of:

- One line with two integers n and w ($1 \leq n \leq 2 \cdot 10^5$, $1 \leq w \leq n$), where n is the number of people in the team and w is the maximum number of people in one row.
- One line with n integers $h_1, \dots, h_n$ ($0 \leq h_i \leq 10^9$ for each i), where h_i is the height of person i .

Output

If it is possible to arrange all people for the photo so that everyone is visible, then output “yes”. Otherwise, output “no”.

Example

Input	Output
5 3 180 160 200 190 150	yes
3 1 150 150 140	no